GUIA DE USO DE GITHUB

CREADOR: ALEJANDRO REALPE MEJIA

INTRODUCCIÓN:
Un factor a tener en cuenta en este repositorio es que al momento de hacer este documento, el repositorio es privado

https://github.com/CIMA-3D/CIMA3D-Prosthesis

Un repositorio privado en GitHub es un espacio de almacenamiento de código y archivos donde solo tú y las personas que tú decidas pueden ver, modificar o descargar el contenido.

A diferencia de los repositorios públicos, el resto de los usuarios de internet no pueden encontrar ni acceder a tu proyecto.

CREAR UN REPOSITORIO PROPIO (OPCIONAL/INFORMATIVO)

Primero un poco de informacion
Crear un repositorio: https://www.youtube.com/watch?v=2hcjEB-Mn7M



Recomendaciones al crear repositorios por parte de github:
…or create a new repository on the command line
echo "# entrenamiento-github" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/alejendroRM/entrenamiento-github.git
git push -u origin main

…or push an existing repository from the command line
git remote add origin https://github.com/alejendroRM/entrenamiento-github.git
git branch -M main
git push -u origin main

PASOS PARA CREAR REPOSITORIO

1. Descargar git y descargar visual studio(no obligatorio pero muy recomendado)
2. Crear una cuenta en github
3. Crear un repositorio en github (se explica en el video tambien)
   3.1. clik en tu perfil → Repositories → boton: NEW
   3.2. Darle el mismo nombre que la carpeta de tu proyecto
   3.3. Cuando crees un nuevo repositorio en la página web de GitHub, asegúrate de **no marcar** la casilla de añadir un archivo `README.md`, licencia o `.gitignore` si vas a subir tu proyecto desde la terminal local. Si GitHub te crea esos archivos en la web y tu computadora está vacía, te dará un conflicto de historiales distintos la primera vez que intentes subir tu código.
   3.4.
4. Crear una carpeta en tu computadora para tu proyecto con el mismo nombre que el punto 3.2
5. Configuración de git
   5.1. Una vez creada la carpeta y los archivos que se quieran subir abrir un terminal
   5.2. Nuevo terminal ctrl+shif+ñ → abrir Git Bash
   5.3. Sigue los siguientes comandos:
   Git crea por defecto la rama master y github la rama main, lo cual crea un conflicto de ramas

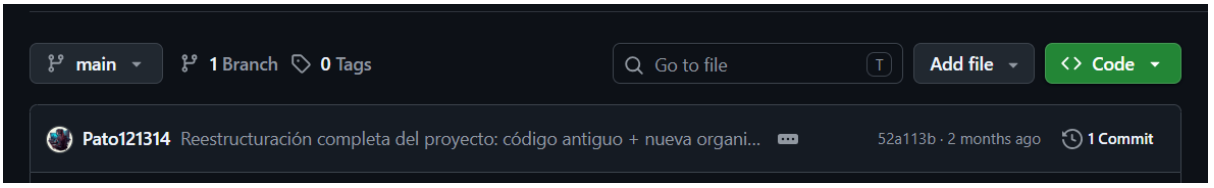
```
git init                                  → Crea la carpeta .git e inicia git
git config --global init.defaultBranch main   → Por defecto crea master y necesitamos main
git branch -M main                        → Esto configura main como la rama principal
git add .                                 → Agrega todos los cambios actuales en un solo pull
git commit -m "pullEjemplo1"              → Prepara
git remote add origin link                → link: de tu repositorio
git push -u origin main                   → Sube el cambio a github
```

CLONAR REPOSITORIO

si al momento de leer este documento el github es público un ignore los pasos del 1

Método 1: clonado desde git bash

1) Te tienen que dar acceso al github como miembro y administrador para poder clonar el repositorio
2) ir a github y localizar el proyecto:



3) En el menú desplegable aparecerá un link:
   https://github.com/CIMA-3D/CIMA3D-Prosthesis.git
4) Copiar el link
5) Dirigirse al git bash en la barra de herramientas de windows
6) localizar la carpeta deseada, donde se quiere clonar el repositorio
   los comandos necesarios para esto son: ls, cd

```
aleja@Kepler MINGW64 ~/Documents
$ ls
 1_KIT_ROBOTIC/              Arduino/                'Mis vídeos'@
 2_UNIVERSIDAD/              CH341SER/               'Plantillas personalizadas de Office'/
'3_PROYECTOS PERSONALES'/    FTMQNA3I84BBUNS.zip     'SOLIDWORKS Downloads'/
 4_Libros/                  'Grabaciones de sonido'/ 'SW Log Files'/
 5_N8N/                      Journals/               Zoom/
 6_Documentos/               MATLAB/                 desktop.ini
 7_Entretenimieto/          'Mi música'@             meshmixer/
 ATENEA_ENTERA.rar          'Mis imágenes'@         'profesor de electronica'/

aleja@Kepler MINGW64 ~/Documents
$ cd 2_UNIVERSIDAD/

aleja@Kepler MINGW64 ~/Documents/2_UNIVERSIDAD
$
```

   aleja@Kepler MINGW64 ~/Documents/2_UNIVERSIDAD/CIMA/codigo26

7) clonamos el repositorio con el comando: git clone mas el link

```
aleja@Kepler MINGW64 ~/Documents/2_UNIVERSIDAD/CIMA/codigo26
$ git clone https://github.com/CIMA-3D/CIMA3D-Prosthesis.git
Cloning into 'CIMA3D-Prosthesis'...
remote: Enumerating objects: 287, done.
remote: Counting objects: 100% (287/287), done.
remote: Compressing objects: 100% (230/230), done.
remote: Total 287 (delta 35), reused 287 (delta 35), pack-reused 0 (from 0)
Receiving objects: 100% (287/287), 20.95 MiB | 10.63 MiB/s, done.
Resolving deltas: 100% (35/35), done.
```

8) Abrimos el visual studio code: code .

Método 2: clonar desde visual studio code

En VS Code:

1. Pulsa `Ctrl+Shift+P` (Windows/Linux) o `Cmd+Shift+P` (macOS).
2. Escribe:
   `Git: Clone`
3. Selecciona **`Clone from GitHub`** si aparece.
4. VS Code debería pedirte iniciar sesión en GitHub.
5. Autoriza VS Code en el navegador con **la cuenta que tiene acceso al repositorio**.
6. Busca:
   `CIMA-3D/CIMA3D-Prosthesis`

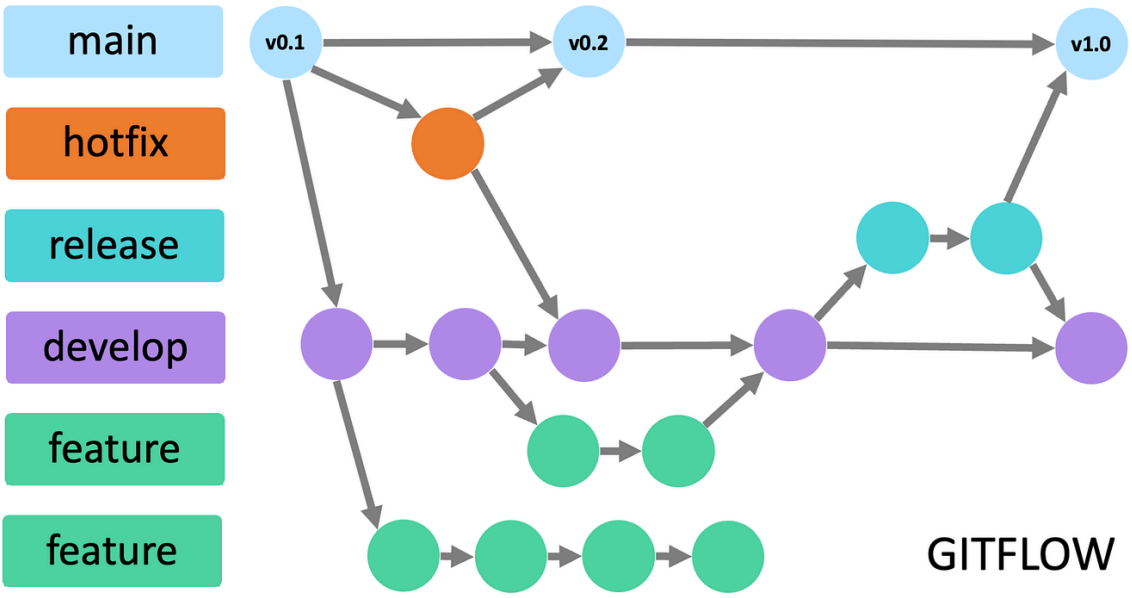
7. Selecciona la carpeta donde quieres guardarlo.
8. Cuando termine, pulsa **Open**.

CREAR UNA RAMA

GitFlow (Modelo estructurado para versiones)

Es un modelo tradicional orientado a proyectos grandes con ciclos de lanzamiento (*releases*) programados. Utiliza dos ramas principales permanentes y varias de apoyo: [1, 2]

- **`main`**: Guarda exclusivamente el historial de versiones oficiales liberadas en producción (etiquetadas con *tags* como `v1.0.0`). [1, 2]
- **`develop`**: Es la rama de integración. Aquí se unen todas las nuevas funcionalidades antes de pasar a producción. [1, 2]
- **Ramas temporales de soporte**:
    - **`feature/*`**: Nacen de `develop` y se fusionan de vuelta en `develop` al terminar la tarea.
    - **`release/*`**: Nacen de `develop` cuando se prepara una nueva versión para salir a producción; se prueban, se corrigen últimos detalles y se fusionan tanto en `main` como en `develop`.
    - **`hotfix/*`**: Nacen de `main` para solucionar un error crítico en producción de forma urgente. Se fusionan en `main` y en `develop`



Convenciones recomendadas para nombrar ramas

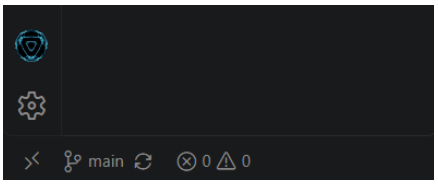
Para mantener orden en el equipo, utiliza prefijos descriptivos en minúsculas: [1]

- `feat/nombre-de-la-caracteristica` (Nuevas funciones)
- `fix/descripcion-del-error` (Corrección de bugs)
- `docs/actualizar-readme` (Cambios en documentación)
- `refactor/limpieza-codigo` (Mejoras de código sin cambiar funcionalidad)
- `hotfix/corrige-fallo-critico` (Arreglos urgentes en producción) [1]

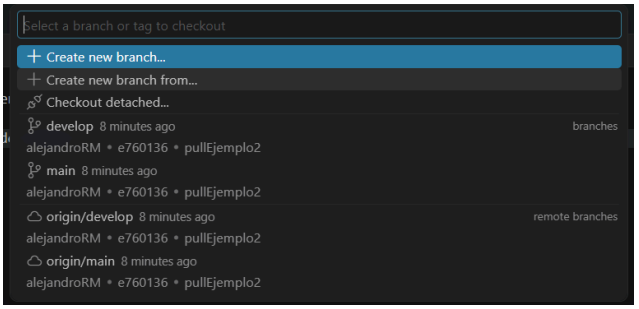
Video tutoria: https://www.youtube.com/watch?v=kEVEt75YZpk

PASOS:

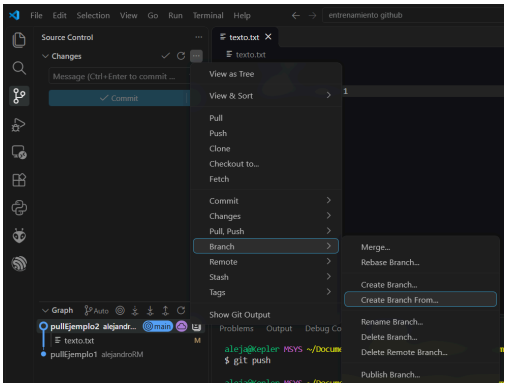
1 verificar en qué rama estamos, en la esquina inferior izquierda se ve como en este caso estamo en la rama main



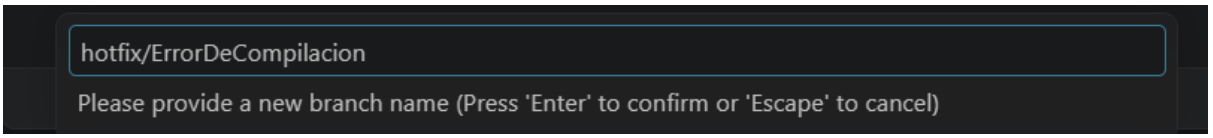
2 si hacemos clic nos aparecerá las ramas que existen



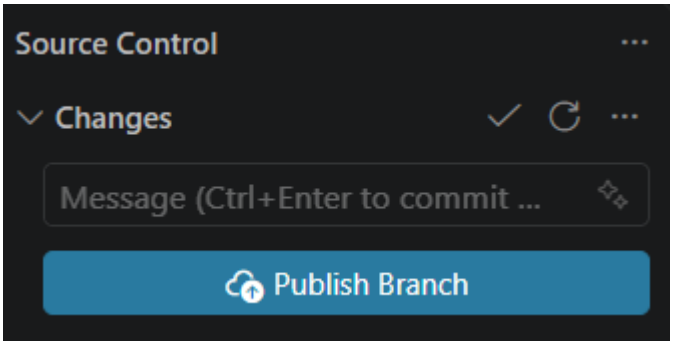
3 En este caso vamos a crear la rama release



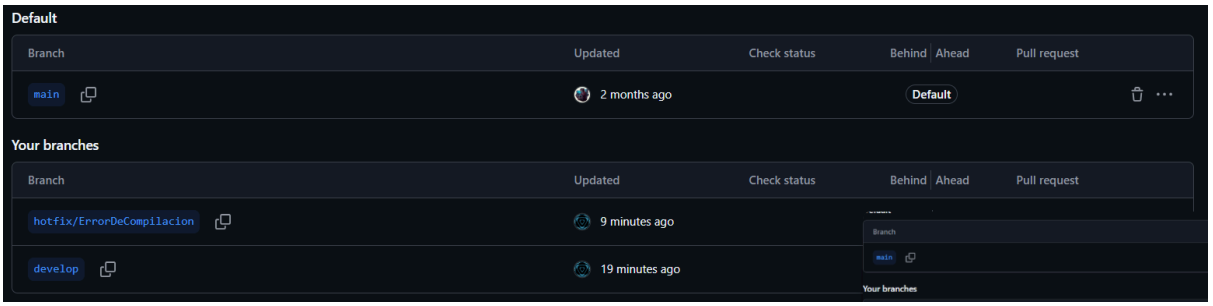
4 seleccionamos la rama desde la cual queremos crear la nueva rama damos enter y le damos un nombre



5 publicamos la rama

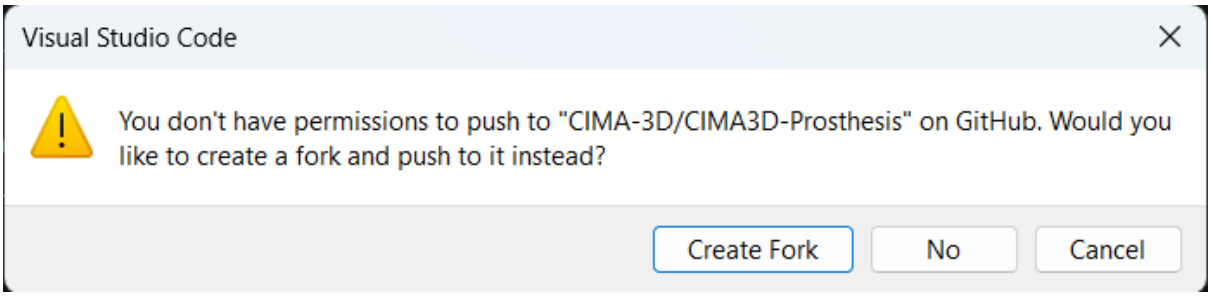


6 verificar en github que se publicaran las ramas



Solución de error por autentificación

si te aparece un mensaje parecido como se recomienda seguir los siguientes pasos pero no son una solución absoluta, el problema lo puede generar muchas variables



Si tu objetivo final es crear una nueva rama no le des en Create Fork
Aunque el repositorio ahora sea público, eso no significa que tengas permisos para modificarlo

GitHub permite que cualquier persona clone un repositorio público, pero solo los colaboradores autorizados pueden subir cambios directamente

1. Comprueba tus permisos en GitHub

Entra en el repositorio:

`https://github.com/CIMA-3D/CIMA3D-Prosthesis`

Ve a:

Settings → Collaborators and teams (o Collaborators, según la configuración).

Comprueba si apareces como colaborador y qué rol tienes:

- Read: solo lectura.
- Triage: gestión de incidencias.
- Write: puedes subir cambios.
- Maintain: puedes administrar el repositorio.
- Admin: control completo.

Lo que necesitamos es que tengas, como mínimo, Write.

2. Comprueba qué cuenta de GitHub está utilizando VS Code

Este punto es especialmente importante.

Puede ocurrir que:

- En el navegador estés conectado con tu cuenta de GitHub del trabajo.
- En Visual Studio Code estés autenticado con otra cuenta personal.
- Git esté utilizando unas credenciales antiguas almacenadas en Windows.

En VS Code:

1. Haz clic en el icono de perfil (abajo a la izquierda).
2. Comprueba qué cuenta de GitHub está conectada.
3. Asegúrate de que es la misma cuenta a la que concedieron permisos en el repositorio.

3. Comprueba la configuración de Git

Abre la terminal de VS Code y ejecuta:

```
None
git remote -v
```

4. Eliminar credenciales incorrectas

- Pulsa `Windows + R`.
- Escribe:

```
None
control /name Microsoft.CredentialManager
```

- Entra en Administrador de credenciales → Credenciales de Windows.
- Busca las credenciales relacionadas con `github.com`.
- Elimina únicamente las credenciales de GitHub que utiliza Git.

Vuelve a VS Code, abre PowerShell en la carpeta del proyecto y ejecuta:

```
None
git push -u origin develop
```

Al no encontrar las credenciales anteriores, Git Credential Manager debería solicitarte autenticarte de nuevo mediante el navegador.

Muy importante: inicia sesión con la cuenta de GitHub que tiene el rol Admin en `CIMA-3D/CIMA3D-Prosthesis`.

Después de autorizar el acceso, vuelve a la terminal y comprueba si se publica la rama.